

The LMTX distribution

keep it simple

context 2022 meeting

The challenge

- How can we provide users with a long term stable LuaMetaT_EX engine that doesn't get messed up by uncontrolled patches?
- How can we ensure a stable current ConT_EXt distribution that works with the engine as it is when that current was published?
- How can users get independence of changes in hardware and the operating system out of their control?
- How can we make sure that the right additional resources that we expect to be present (with a given version) are kept with the engine and distribution?
- How can we make installing and running the lot mainly a matter of running the single independent binary in a well defined T_EX tree?
- How can we stay stable and stay free from these ever changing fashions, pseudo standards, software politics, etc.?
- In other words how can we best distribute ConT_EXt LMTX best?

Running

- We only need one binary: `luametatex`.
- We run ConT_EXt with `context` which is a link to the binary.
- We run other scripts with `mtxrun` which is a link to the binary.
- These runners have `context.lua` and `mtxrun.lua` (with suffix) in the same (binary) path.
- So three files and two links is all what is needed.
- We assume a standardized setup. One can divert but we're not going to waste time on alternatives.
- Here the resources are all located relative to the binary. This is a proven setup (minimals).

Directory structure

This is how one can install it:

```
always    /data/context/tex/texmf/  
always    /data/context/tex/texmf-<platform>/bin/  
always    /data/context/tex/texmf-context/  
  
optional  /data/context/tex/texmf-local/  
optional  /data/context/tex/texmf-fonts/  
optional  /data/context/tex/texmf-project/
```

After running:

```
1 mtxrun --generate  
2 mtxrun --script font --reload
```

This should work:

```
1 context [--autopdf] --global welcome-to-context
```

It picks up a file from the tree and runs it on the local path.

A solution

- Already years ago (and also recently when we met) Mojca and I discussed this but we never came to implement it. Of course we blame COVID.
- It boils down to having several **repositories** from which we install:
- The **ConT_EXt distribution**, including the source of LuaMetaT_EX. Basically this is the already present mirror.
- A repository that has (mostly) **reliable fonts** that we know work as we expect and don't get replaced without them being tested.
- A repository with stable and quality assured **reliable third party modules**.
- A repository with **binaries** that match the latest version. Users can always generate their own if needed.
- **Snapshots** of working combination of these that can get installed manually or via an installer.

- Basically we're talking of what is (and has always) been the minimal but maintained by some more people. New is that we provide the means to compile the engine, which is the only program needed.
- We stay far from dependencies so we have an independent source tree. There is the possibility to load libraries, but we provide only very few small optional interfaces. Additional ones can be made outside the source tree but will not be distributed.
- For the engine as well as the optional libraries we can set up the compile farm but for the later we will not enter a frequent update mode: we need little.

Compiling

- Are we ready to distribute the source? Last year we didn't release because one important condition was not met. But now we can decide to release version 1.00!
- Although there are plenty of places where I want to go a bit further, most is done.
- Can we expect users to compile binaries? The answer for a complete T_EXLive setup is 'no' but for what we have here is can work well.
- In due time we can have recipes on the garden but it should work more or less out of the box: linux and MS Windows demo.
- Michal Vlasák made a prototype of setup for compiling some libraries.
- We need to get LMTX into the next T_EXLive. Because we don't rely on the build infrastructure Mojca probably has to provide the binaries from the farm.